

Project Proposal (Week 2)

Energy-Aware Ventilation & Climate Control

Kasper Axelsson

Mohamd Amar Chms Aldden

D7065E

Summary

Team	Kasper Axelsson, Mohamd Amar Chms Aldden
Use case	Energy-aware ventilation & climate control
Pitch	Infer who is in each room from CO ₂ and temperature alone, anticipate when a room is about to fill, and condition it in advance—keeping air quality and comfort high while cutting energy.
Repository	https://github.com/MisterD0ctor/d7065e-project

1 Use case, and why

This project controls ventilation and heating in a BuildSim office floor. It was chosen because it has a genuine tension—air quality and comfort pull one way, energy the other—which makes the autonomous decision interesting, and because BuildSim provides the rooms and the people to work with.

What makes the decision more than a thermostat is *lag*. A room does not respond quickly: flushing accumulated CO₂ takes on the order of the air-exchange time constant, tens of minutes at normal flow rates, and the thermal response of the room and its envelope is slower still. By the time a sensor shows that a conference room has filled up, the system is already behind, and catching up means running the fans hard—exactly the expensive, noisy behaviour we want to avoid. A controller that knows a room is *about to* fill can start early and gently instead. This is the reason occupancy prediction is worth doing at all, and it is what the evaluation in Section 7 is built to test.

2 Scope: no occupancy sensor

We deliberately do **not** give the controller a per-room occupancy sensor. Real offices rarely have reliable people-counting in every room, and a project that assumes one has solved the easy half of the problem by assumption. Instead the controller sees only what a realistic installation would have, temperature and CO₂, and has to work out occupancy for itself.

The simulator still generates a ground-truth head-count per room, because it needs one to drive the physics, but that signal is *not* exposed to the controller. It is used only as the reference for measuring how good our occupancy estimates are. If the estimator turns out to be infeasible, exposing occupancy directly is our fallback, and we would report it as such.

3 Scenario

A meeting fills a conference room. CO₂ begins to climb; from the rate of climb the system infers that several people have entered, and it increases ventilation, holding CO₂ below 1000 ppm.

Meanwhile the recurring pattern learned for that room says the 13:00 slot is usually busy, so on the following day ventilation and heating ramp up shortly *before* 13:00 and the room is already at setpoint when people arrive. When the room empties, the system detects the decay and falls back to a low-energy setback. The same loop runs in every room.

4 What will be built (rough sketch)

- **Sensors:** temperature and CO₂ per room, registered with BuildSim, reported at realistic sampling intervals and with noise.
- **Autonomous decision-maker:** estimates current occupancy, predicts near-future occupancy, and sets ventilation level and heating setpoint per room.
- **Actuators:** HVAC setpoint and ventilation damper, writing state back to BuildSim.
- **Data pipeline / storage:** collect sensor readings, store recent history, and serve it to the autonomous service, the training job, and the dashboard.
- **Dashboard:** overlay measured state, estimated occupancy, and decisions on the BuildSim 3D view.

Each major component (sensors, autonomous service, actuators, storage, and dashboard) runs as a *separate process* communicating through defined interfaces, and talks to BuildSim through its API. Transport is REST to begin with, on the understanding that we may have to move to a message broker as the number of rooms grows (Section 8).

5 Simulating the building

BuildSim provides the rooms; the physics and the people are simulated as follows.

Physical processes. A simple per-room model: temperature from a heat balance (envelope loss, body heat, HVAC), and CO₂ from a mass balance, rising with the number of people and decaying towards outdoor concentration with ventilation. Both are first-order, with time constants chosen to be plausible for an office room, since the premise of the whole project rests on those constants not being negligible.

People movement (occupancy). Sample a head-count per room conditioned on room type and time of day: offices steady through the day, the fika room busy around 11:30, conference rooms full during meetings, empty at night. The pattern is regular enough to be learnable but carries day-to-day variation, so that prediction is useful without being trivial.

The models are intentionally simple, but **actuator actions must influence future sensor readings** so that the control loop closes.

6 The autonomous part

The decision-maker runs in three stages per room.

1. **Estimate now.** Invert the CO₂ mass balance: given the measured concentration, its rate of change, and the ventilation flow we ourselves commanded, solve for the head-count that explains it. This is a physical estimate rather than a learned one, so it gives us something working before any model is trained.
2. **Predict soon.** Learn each room's occupancy pattern from stored history—time of day, day of week, and the recent estimate—and predict the head-count one lead time ahead, where the lead time is set by the room's measured response time. We start small (a per-room time-of-day profile, then gradient boosting if that proves too blunt) rather than reaching for anything large.
3. **Act.** Feed predicted occupancy and current state to a controller that sets fan level and heating setpoint, biased towards starting early and gently rather than reacting hard.

Stages 1 and 3 alone are already a complete working system. Stage 2 is the part that has to earn its place, which is what the evaluation is for.

7 Evaluation and baselines

We compare our controller against three references over the same simulated week:

1. **Constant ventilation** at design flow. Good air quality, bad energy; the upper bound on comfort.
2. **Reactive threshold control** on measured CO₂. The honest competitor, and what most real buildings actually do.
3. **Oracle:** our controller given the true future occupancy instead of a prediction. The upper bound on what anticipation could buy.

Reported per room and per day: minutes above 1000 ppm; minutes outside the comfort band while occupied; an energy proxy combining fan power and delivered heat; and, for the estimator, error against the simulator’s ground-truth head-count. The gap between the reactive baseline and the oracle bounds how much prediction is worth here. If that gap turns out to be small, that is a result worth reporting rather than a failure to hide.

8 Biggest unknowns & risks

- **Is occupancy identifiable from CO₂ alone?** Telling two people from four may fall within sensor noise, particularly at high ventilation rates where the concentration signal is small. Mitigation: fall back to a coarser target—empty / few / many—which is likely all the controller needs.
- **Does anticipation actually pay off?** It only helps if the simulated lag is comparable to the prediction lead time. If our rooms respond within a few minutes, the reactive baseline will match us and the predictive layer is dead weight. Mitigation: pick time constants from published figures for office rooms and state them explicitly.
- **Does REST keep up?** Polling every room over REST may not scale as rooms are added, and we may need broker-based transport instead.
- **Is the room physics realistic enough** for the decisions to mean anything, given that we are evaluating a controller against a model we wrote ourselves.

9 First step & rough plan

The first step is *one room working end-to-end*—one set of sensors → controller → actuator → BuildSim—to prove the loop closes, with the trivial reactive controller in place. Everything after that is layered onto a loop that already runs.

Week	Target
3	One room end-to-end, reactive control, loop closed.
4	All rooms; storage and logging; occupancy simulation.
5	CO ₂ -based occupancy estimator, scored against ground truth.
6	Predictive layer and the three baselines; first numbers.
7	Dashboard, failure handling, evaluation runs.
8	Demo.